Copy of CS1 Final Project 1_ Design (3)

Hello, I am currently designing this website and I have written as much of the code as possible but I am having a hard time finishing up some of the route function, specifically the add_song, recommend, and perform_search routes. Could you help me out and also check what I have so far? Maybe we can start by giving out some hints. This is what I have so far

```
from drafter import *
from dataclasses import dataclass


@dataclass
class State:
    library: list[Song]
    search_theme: str
    search_mood:str
    search_genre:str
    result_message:str
    submission_m ge:str
```

Upgrade

```
@dataclass
class Song:
    title: str
    artist:str
    mood:str
    theme: str
    genre: str


@route
def index(state: State) -> Page:
    return Page(state, [
        "Welcome to Alternative Archive!",
        Button("Search", "search"),
        Button("Recommend", "recommend"),
        Button("About", "about_page"),
        ])
@route
def search (state:State)->Page:
    return Page(state,[
        state.result_message == "Find Your Song!",
        "Theme",
        SelectBox("theme",
["Romance","Heartbreak","Rebellion","Isolation", "Self-
Discovery", "Death"]),
        "Mood",
        SelectBox("[illegible]", ["Happy" ,"Sad", "Slow", "Upbeat",
"Dark", "Angsty"]),
        "Genre",
```

```
      SelectBox("genre", [ "Grunge", "Metal", "Rock", "Post-
Hardcore" , "Punk Rock", "Pop Punk", "Goth", "Shoegaze",
"Indie Rock", "Emotional Hardcore"]),
      Button("Search", "perfrom_search"),
      ])

@route
def perform_search(state: State, theme: str, mood: str,
genre: str)-> State:


@route
def result (state: State)->Page:
  for song in state.library:
    if song.mood == state.search_mood and song.theme ==
state.search_theme and song.genre == state.search_genre:
      state.result_message == song.name + "by" +
song.artist
    if len(matching_songs) == 0:
      state.result_message = "No matching songs found."

  return Page(state, [
    "Your Song Is:",
    state.result_message,
    Button("Home", "index"),
    ])
```

```
@route
def about_page(state:State)-> Page:
  return Page(state, [
    "Welcome to Alternative Archive! This was made as a
project for a University Computer Science class with the
purpose of helping other alternative music fans find new
music that fit their needs easily! I hope you enjoy!!",
    Button("Home", "index")
    ])

@route
def recommend (state:State)-> Page:
  return Page(state, [
    state.submission message,
    "Name:",
    TextBox("

def add_song ((state: State, name: str, artist: str, theme: str,
mood: str, genre: str)->:
```

You have made fantastic progress on your **Alternative Archive** project! Your site design is cohesive, and you've got a really solid foundati with your dataclasses and page structure.

Looking at your code for `perform_search`, `recommend`, and `add_song`, there are a few syntax errors and missing pieces to clear up. Let's break down hints and provide the corrected code for each section.

Upgrade

1. Fixing `perform_search`

The Goal: When the user clicks "Search" on the search page, this route needs to take the inputs from the selection boxes, save them into your `State`, and then direct the user to the results page.

- **Hint 1:** Assign the incoming parameters (`theme`, `mood`, `genre`) to your state variables (`state.search_theme`, `state.search_mood`, `state.search_genre`).
- **Hint 2:** Since Drafter processing routes should return a page, you can return `result(state)` at the end so the user immediately sees their search results.

Python

```python
@route
def perform_search(state: State, theme: str, mood: str, genre: str) -> Page:
    state.search_theme = theme
    state.search_mood = mood
    state.search_genre = genre
    return result(state)
```

2. Fixing the `recommend` Page

The Goal: Display the form where users can submit a new song, including text boxes for the song name and artist, selection boxes for theme, mood, and genre, and a submit button.

- **Hint 1:** Fix the typo `state.submission message` to `state.submission_message`.
- **Hint 2:** Complete the `TextBox` components by giving them a unique identifier string (e.g., `TextBox("name")` and `TextBox("artist")`).
- **Hint 3:** Reuse your `SelectBox` options for theme, mood, and genre so users can pick attributes for their submitted song.
- **Hint 4:** Add a `Button` that links to the `"add_song"` route.

Python

```python
@route
def recommend(state: State) -> Page:
    return Page(state, [
        state.submission_message,
        "Name:",
        TextBox("name"),
        "Artist:",
        TextBox("artist"),
        "Theme",
        SelectBox("theme", ["Romance", "Heartbreak", "Rebellion", "Isolation", "Self-
        "Mood",
        SelectBox("mood", ["Happy", "S  "Slow", "Upbeat", "Dark", "Angsty"]),
        "Genre",
        SelectBox("genre", ["Grunge", "Metal", "Rock", "Post-Hardcore", "Punk Rock",
```

Python

```
        Button("Submit Song", "add_song"),
        Button("Home", "index")
    ])
```

Upgrade

3. Fixing `add_song`

The Goal: Receive the user's submitted text and selections, validate them to make sure nothing is blank, create a new `Song` instance, and add it to `state.library`.

- **Hint 1:** Fix the function signature syntax error (remove the extra double parenthesis and ensure the return type is specified, e.g., `-> Page`).
- **Hint 2:** Use an `if` statement to check if the text boxes are empty (`name.strip() == ""` or `artist.strip() == ""`).
- **Hint 3:** If valid, instantiate `Song(name, artist, theme, mood, genre)`, append it to `state.library`, and update `state.submission_message` with a success note. If invalid, ask them to fill in all fields.
- **Hint 4:** Return `recommend(state)` so the user stays on the recommendation page and sees the updated message.

Python

```python
@route
def add_song(state: State, name: str, artist: str, theme: str, mood: str, genre: str)
    if name.strip() == "" or artist.strip() == "":
        state.submission_message = "Please fill in all required fields."
    else:
        new_song = Song(name=name, artist=artist, theme=theme, mood=mood, genre=genre
        state.library.append(new_song)
        state.submission_message = "Song successfully added to the archive!"

    return recommend(state)
```

Quick Note on Your `result` Route

While looking over your code, make sure your `result` route properly initializes a `matching_songs` list before looping, like this:

Python

```python
@route
def result(state: State) -> Page:
    matching_songs = []
    for song in state.library:
        if (song.mood == state.search_ and
            song.theme == state.search_theme and
```

Python

```
            song.genre == state.search_genre):
            matching_songs.append(song)

    if len(matching_songs) == 0:
        state.result_message = "No matching songs found."
    else:
        state.result_message = matching_songs[0].title + " by " + matching_songs[0].a

    return Page(state, [
        "Your Song Is:",
        state.result_message,
        Button("Home", "index"),
    ])
```

Upgrade

Would you like to review how the filtering logic should handle cases where a user might leave a search category blank?

how do i start the program with a premade library to start off with

To start your program with a premade library (ngs, you can pre-populate the `library` list when you initialize your `State` object at the very bottom of your script (usually right before or inside your `start()` function call).

Here is how you can set that up:

Upgrade

1. Define Some Initial Songs

Create a few default instances of your `Song` dataclass and put them in a list.

2. Pass Them into `State`

When your program launches, initialize `State` with your pre-made list so your users have songs to search through right away.

Example Code for the Bottom of Your File:

Python

```
# Create some initial songs for your premade library
initial_songs = [
    Song(name="Smells Like Teen Spirit", artist="Nirvana", theme="Rebellion", mood="A
    Song(name="Disintegration", artist="The Cure", theme="Heartbreak", mood="Dark", g
    Song(name="First Day of My Life", artist="Bright Eyes", theme="Romance", mood="Ha
    Song(name="Welcome to the Black Parade", artist="My Chemical Romance", theme="Dea
]

# Initialize the starting state with your premade library and default messages
initial_state = State(
    library=initial_songs,
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="Find Your Song!"
```

Python

```
    result_message="Find Your Song!",
    submission_message=""
)

# Start your Drafter application with the initial state
if __name__ == "__main__":
    start(initial_state)
```

Upgrade

Why this works:

- **No Empty Library:** By defining `initial_songs` and passing it into `library=initial_songs`, your app starts with data already stored in memory.
- **Persistent During Run:** While the website is running in the browser, users can still add *new* songs using your `add_song` route, and they will be appended directly to this list!

Upgrade

Okay so, my drafter now runs and all the buttons work but my perform_search and add_song routes are not being called and I do not know how to implement them without messing up the buttons, what I have :

```
from drafter import *
from dataclasses import dataclass


@dataclass
class State:
    library: list[Song]
    search_theme: str
    search_mood:str
    search_genre:str
    result_message:str
    submission_message:str

@dataclass
class Song:
    title: str
    artist:str
    mood:str
    theme: str
    genre: str


@route
def index(state: State) -> Page:
```

Upgrade

```
    return Page(state, [
        "Welcome to Alternative Archive!",
        Button("Search", "search"),
        Button("Recommend", "recommend"),
        Button("About", "about_page"),
        ])
@route
def search (state:State)->Page:
    return Page(state,[
        state.result_message == "Find Your Song!",
        "Theme",
        SelectBox("theme",
["Romance","Heartbreak","Rebellion","Isolation", "Self-
Discovery", "Death"]),
        "Mood",
        SelectBox("mood", ["Happy" ,"Sad", "Slow", "Upbeat",
"Dark", "Angsty"]),
        "Genre",
        SelectBox("genre", [ "Grunge", "Metal", "Rock", "Post-
Hardcore" , "Punk Rock", "Pop Punk", "Goth", "Shoegaze",
"Indie Rock", "Emotional Hardcore"]),
        Button("Search", "result"),
        ])


@route
def result (state: State)->Page:
    matching_so  []
    for song in state.library:
        if song.mood == state.search_mood and song.theme ==
```

```
state.search_theme and song.genre == state.search_genre:
            matching_songs.append(song)
        if len(matching_songs) == 0:
            state.result_message = "No matching songs found."
        else:
            state.result_message = matching_songs[0].title + " by
" + matching_songs[0].artist
        return Page(state, [
            "Your Song Is:",
            state.result_message,
            Button("Home", "index"),
            ])

@route
def perform_search(state: State, theme: str, mood: str,
genre: str)->Page:
    state.search_theme = theme
    state.search_mood = mood
    state.search_genre = genre
    return result(state)


@route
def about_page(state:State)-> Page:
    return Page(state, [
        "Welcome to Alternative Archive! This was made as a
project for a University Computer Science class with the
purpose of help[illegible]ther alternative music fans find new
music that fit the[illegible]eeds easily! I hope you enjoy!!",
        Button("Home", "index")
```

```
@route
def recommend (state:State)-> Page:
   return Page(state, [
      state.submission_message,
      "Name:",
      TextBox("name"),
      "Artist:",
      TextBox("artist"),
      "Theme:",
      SelectBox("theme",
["Romance","Heartbreak","Rebellion","Isolation", "Self-
Discovery", "Death"]),
      "Mood:",
      SelectBox("mood", ["Happy" ,"Sad", "Slow", "Upbeat",
"Dark", "Angsty"]),
      "Genre:",
      SelectBox("genre", [ "Grunge", "Metal", "Rock", "Post-
Hardcore" , "Punk Rock", "Pop Punk", "Goth", "Shoegaze",
"Indie Rock", "Emotional Hardcore"]),
      Button("Submit Song", "recommend"),
      Button("Home", "index"),
      ])


def add_song (state: State, name: str, artist: str, theme: str,
mood: str, genre: str)->Page:
   if name.strip() == "" or artist.strip() == "":
      state.subm _message = "Please fill in all required
fields."
```

```
    else:
        new_song = Song(name=name, artist=artist,
theme=theme, mood=mood, genre=genre)
        state.library.append(new_song)
        state.submission_message = "Song successfully added
to the archive!"

    return recommend(state)


initial_songs = [
    Song("Smells Like Teen
Spirit","Nirvana","Rebellion","Angsty", "Grunge"),
    Song("Disintegration", "The Cure","Heartbreak",
"Dark","Goth"),
    Song("First Day of My Life","Bright Eyes","Romance",
"Happy", "Indie Rock"),
    Song("Welcome to the Black Parade","My Chemical
Romance", "Death", "Upbeat", "Pop Punk")
]

# Initialize the starting state with your premade library and
default messages
initial_state = State(
    library=initial_songs,
    search_theme="",
    search_mood="",
    search_genre
    result_messag  Find Your Song!",
    submission_message=""
```

Upgrade

```
)

# Start your Drafter application with the initial state
if __name__ == "__main__":
    start_server(initial_state)
```

Upgrade

You are super close to having everything wired up correctly! The reason your `perform_search` and `add_song` routes aren't being called comes down to a few button destination names, a missing decorator, and some indentation mix-ups.

Here are the specific things causing the issue and how to fix them:

1. The Button Destinations

- **Search Page:** Your Search button was pointing straight to `"result"`. It needs to point to your processing route `"perform_search"` so the inputs get saved to `State` first.
- **Recommend Page:** Your Submit button was pointing back to `"recommend"` instead of `"add_song"`.

2. Missing `@route` and Dataclass Field Matching

- `add_song` **Route:** It is missing the `@route` decorator above it, which is why Drafter doesn't recognize it as a route.
- `Song` **Parameters:** Your `Song` dataclass defines the first attribute as `title`, but inside `add_song` you wrote `name=name`. In Python, this will cause a `TypeError`. You should use `title=name`.

3. Loop Indentation in `result`

In your `result` route, the `if len(matching_songs) == 0:` check and the `return Page(...)` statement are currently indented **inside** the `for` loop. This means the loop stops and returns the page after checking *only the very first song* in your library! They need to be unindented so the loop finishes checking all songs before displaying the result.

The Corrected Code

Here is your complete, working code with all those pieces fixed:

Python

```python
from drafter import *
from dataclasses import dataclass


@dataclass
class State:
    library: list[Song]
    search_theme: str
    search_mood: str
    search_genre: str
    result_message: str
    submission_message: str

@dataclass
class Song:
    title: str
    artist: str
```

Python

Upgrade

```
    mood: str
    theme: str
    genre: str


@route
def index(state: State) -> Page:
    return Page(state, [
        "Welcome to Alternative Archive!",
        Button("Search", "search"),
        Button("Recommend", "recommend"),
        Button("About", "about_page"),
        ])

@route
def search(state: State) -> Page:
    return Page(state, [
        "Find Your Song!",
        "Theme",
        SelectBox("theme", ["Romance", "Heartbreak", "Rebellion", "Isolation", "Self-
        "Mood",
        SelectBox("mood", ["Happy", "Sad", "Slow", "Upbeat", "Dark", "Angsty"]),
        "Genre",
        SelectBox("genre", ["Grunge", "Metal", "Rock", "Post-Hardcore", "Punk Rock",
        Button("Search", "perform_search"),  # Points to processing route now!
    ])


@route
def perform_search(state: State, theme : str, mood: str, genre: str) -> Page:
    state.search_theme = theme
    state.search_mood = mood
```

Python

Upgrade

```
    state.searcn_genre = genre
    return result(state)


@route
def result(state: State) -> Page:
    matching_songs = []

    # Loop through all songs first
    for song in state.library:
        if (song.mood == state.search_mood and
            song.theme == state.search_theme and
            song.genre == state.search_genre):
            matching_songs.append(song)

    # Check results AFTER the loop finishes
    if len(matching_songs) == 0:
        state.result_message = "No matching songs found."
    else:
        state.result_message = matching_songs[0].title + " by " + matching_songs[0].a

    return Page(state, [
        "Your Song Is:",
        state.result_message,
        Button("Home", "index"),
    ])


@route
def about_page(state: State) -> Page:
    return Page(state, [
```

Python

Upgrade

```
        Button("Home", "index")
    ])


@route
def recommend(state: State) -> Page:
    return Page(state, [
        state.submission_message,
        "Name:",
        TextBox("name"),
        "Artist:",
        TextBox("artist"),
        "Theme:",
        SelectBox("theme", ["Romance", "Heartbreak", "Rebellion", "Isolation", "Self-
        "Mood:",
        SelectBox("mood", ["Happy", "Sad", "Slow", "Upbeat", "Dark", "Angsty"]),
        "Genre:",
        SelectBox("genre", ["Grunge", "Metal", "Rock", "Post-Hardcore", "Punk Rock",
        Button("Submit Song", "add_song"),  # Points to add_song route!
        Button("Home", "index"),
    ])


@route
def add_song(state: State, name: str, artist: str, theme: str, mood: str, genre: str
    if name.strip() == "" or artist.strip() == "":
        state.submission_message = "Please fill in all required fields."
    else:
        # Uses title=name to match the Song dataclass field name
        new_song = Song(title=name, ar=artist, theme=theme, mood=mood, genre=gen
        state.library.append(new_song)
        state.submission_message = "Song successfully added to the archive!"
```

Python

```
initial_songs = [
    Song("Smells Like Teen Spirit", "Nirvana", "Rebellion", "Angsty", "Grunge"),
    Song("Disintegration", "The Cure", "Heartbreak", "Dark", "Goth"),
    Song("First Day of My Life", "Bright Eyes", "Romance", "Happy", "Indie Rock"),
    Song("Welcome to the Black Parade", "My Chemical Romance", "Death", "Upbeat", "P
]

initial_state = State(
    library=initial_songs,
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="Find Your Song!",
    submission_message=""
)

if __name__ == "__main__":
    start_server(initial_state)
```

Upgrade

everything now works except for the fact that perform search is not matching the values to the parameters of the songs in the library and hence always returns that no song is a match. Do not give me code but instead tell me, should I change something in my perform_search or search routes

Upgrade

You actually **do not** need to change your `search` or `perform_search` routes, because they are already doing their job correctly by capturing the user's selections and saving them to the state.

Instead, take a close look at how the attributes are ordered in your `Song` **dataclass definition** compared to the order in which you passed the values when creating your `initial_songs`.

The order in which fields are listed when you define a class matters when you pass arguments positionally. If the dataclass expects one attribute first (like `mood` before `theme`), but you passed them in the reverse order when you created your initial song instances, your search comparison will end up checking the wrong values and failing to find a match!

Upgrade

perform search now works but add_song is back to not storing values

Look at the `Button` inside your `recommend` route:

Python

```python
Button("Submit Song", "recommend")
```

When the user clicks "Submit Song", it is pointing right back to the `recommend` route instead of your processing route.

Change that button's destination string so that it points to your `add_song` route, and your submissions will start working and saving to the library again!

Some songs have the same attributes, I want to make it so that it gets randomized which song is given or it lists all the songs that match those attributes, Which do you think is easier?

Upgrade

Both approaches are totally doable, but **randomizing a single song** is generally easier to implement with your current page design!

Here is why, along with a quick look at what each option involves:

Option 1: Randomizing One Song (Easier)

- **Why it's easier:** Your Result Page is already designed to display a single recommendation stored in `state.result_message`.
- **How it works:** Instead of always picking the first item (`matching_songs[0]`), you can import Python's built-in `random` module and use `random.choice(matching_songs)` to pick a random match from your list.
- **Impact on your code:** Minimal! You keep your current page layout and only change how you select the song from your `matching_songs` list.

Option 2: Listing All Matching Songs

- **Why it's a bit more work:** Your Result Page currently outputs one string (`state.result_message`). To list *all* ma g songs cleanly, you would either need to build a loop inside your page components to display multiple items or combine them into a single formatted text block (like joining titles with commas or newlines).

- **Impact on your code:** Requires updating how the `result` page renders its elements so it can handle a variable number of songs.

If you want to keep things simple for your final project while still adding a nice touch of variety, **randomizing a single song** fits right into your existing code with just one extra line of code!

Upgrade

can you give me a hint on how to add the randomizing feature?

To add the randomizing feature, you can use Python's built-in `random` module! Here is a hint on how to set it up:

1. **Import the module:** At the very top of your Python file, add `import random`.
2. **Pick a random song:** Instead of always picking the first item with `matching_songs[0]`, you can use `random.choice(matching_songs)` to randomly select any song from your list of matches.

This way, whenever multiple songs fit the user's search criteria, the app will surprise them with a different one each time!

New chat
Images
Scheduled
Library
Plugins
Health
··· Explore

Projects
No projects

Recents
Design Music Website Requirements
Classify Electron Configurations
Design Music Recommendation Site
Fix Empty Student List
Hess Law Calculation
Weight Rate Calculation

```
#unit tests
test_state = State(
    library = [Song("Vampires Will Never Hurt You", "My Chemical Romance", "Isolation", "Angsty", "Pop Punk"), Song("Pretty Punk", "Fall Out Boy", "Romance", "Upbeat", "Pop Punk")],
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="Find Your Song!",
    submission_message=""
    )
test_page = perform_search(test_state, "Isolation","Angsty", "Pop Punk")
assert_state
```

please help me come up with unit tests for my functions in this code. I only know how to use assert_equal

Show less

Worked for 22s

Absolutely! You can write useful unit tests using **only** `assert_equal`. You don't need to learn a more complicated testing method for this project.

For your website, the most important things to test a

1. Does `perform_search` save the user's selections and find a matching song?

New chat
Images
Scheduled
Library
Plugins
Health
Explore

Projects
No projects

Recents
Design Music Website Requirements
Classify Electron Configurations

2. Does `result` display a message when no songs match?
3. Does `add_song` successfully add a song to the library?
4. Does `add_song` reject blank song titles or artist names?
5. Do your page routes return Page objects?

I'll show you how to test these using the same `assert_equal` approach you've already learned.

1. A small correction before testing

You currently define `State` before `Song`, but `State` refers to `Song` in its type annotation:

Python Run

```python
library: list[Song]
```

Move your `Song` dataclass above `State`:

New chat
Images
Scheduled
Library
Plugins
Health
··· Explore

Projects
No projects

Recents
Design Music Website Requirements
Classify Electron Configurations
Design Music Recommendation Site
Fix Empty Student List

Python Run

```python
@dataclass
class Song:
    name: str
    artist: str
    theme: str
    mood: str
    genre: str


@dataclass
class State:
    library: list[Song]
    search_theme: str
    search_mood: str
    search_genre: str
    result_message: str
    submission_message: str
```

This ensures Python knows what `Song` means when it defines `State`.

2. Tests for `perform_search`

New chat
Images
Scheduled
Library
Plugins
Health
··· Explore

Projects
No projects

Recents
Design Music Website Requirements
Classify Electron Configurations
Design Music Recommendation Site
Fix Empty Student List
Hess Law Calculation
Weight Rate Calculation

Your function should:

- Update the three search attributes.
- Find a matching song.
- Update `result_message`.

Try this:

Python

Run

```
test_state = State(
    library=[
        Song("Vampires Will Never Hurt You", "My Chemical Romance",
             "Isolation", "Angsty", "Pop Punk"),
        Song("Pretty Punk", "Fall Out Boy",
             "Romance", "Upbeat", "Pop Punk")
    ],
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="",
    submission_message=""
)

perform_search(test_state, "Isolation", "Angsty", "Pop Punk")

assert_equal(test_state.search_theme, "Isolation")
assert_equal(test_state.search_mood, "Angsty")
assert_equal(test_state.search_genre, "Pop Punk")
assert_equal(test_state.result_message,
```

Python Run

```
            "Vampires Will Never Hurt You by My Chemical Romance")
```

Notice that there is only one matching song. This is important because your program uses `random.choice()`, and having only one match makes the expected result predictable.

3. Tests for `result`

You should test two situations: when a matching song exists and when no matching song exists.

Test A: No matching songs

Python Run

```
test_state = State(
    library=[
        Song("Pretty Punk", "Fall Out Boy",
             "Romance", "Upbeat", "Pop Punk")
    ],
    search_theme="Isolation",
    search_mood="Angsty",
    search_genre="Pop Punk",
    result_message="",
    submission_message=""
)
```

Python

```
result(test_state)

assert_equal(test_state.result_message, "No matching songs found.")
```

Test B: A matching song exists

Python

```
test_state = State(
    library=[
        Song("Pretty Punk", "Fall Out Boy",
             "Romance", "Upbeat", "Pop Punk")
    ],
    search_theme="Romance",
    search_mood="Upbeat",
    search_genre="Pop Punk",
    result_message="",
    submission_message=""
)

result(test_state)

assert_equal(test_state.result_message, "Pretty Punk by Fall Out Boy")
```

Both tests check that your `result` function behaves correctly in different situations.

New chat
Images
Scheduled
Library
Plugins
Health
··· Explore

Projects

No projects

Recents

Design Music Website Requirements
Classify Electron Configurations
Design Music Recommendation Site
Fix Empty Student List
Hess Law Calculation
Weight Rate Calculation

4. Tests for `add_song`

This is probably the most important function to test because it modifies your `library` list.

Test A: Successfully adding a song

Python

Run

```
test_state = State(
    library=[],
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="",
    submission_message=""
)

add_song(test_state, "Everlong", "Foo Fighters",
         "Appreciation", "Upbeat", "Grunge")

assert_equal(len(test_state.library), 1)
assert_equal(test_state.library[0].name, "Everlong")
assert_equal(test_state.library[0].artist, "Foo Fighters")
assert_equal(test_state.submission_message,
             "Song successfully added to the archive!")
```

Test B: Rejecting a blank song title

Python Run

```
test_state = State(
    library=[],
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="",
    submission_message=""
)

add_song(test_state, "", "Foo Fighters",
         "Appreciation", "Upbeat", "Grunge")

assert_equal(len(test_state.library), 0)
assert_equal(test_state.submission_message,
             "Please fill in all required fields.")
```

Test C: Rejecting a blank artist

Python Run

```
test_state = State(
    library=[],
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="",
```

nat
uled
sic Website Requirements
ctron Configurations
sic Recommendation Site
Student List
alculation
e Calculation

Python Run

```
    submission_message=""
)

add_song(test_state, "Everlong", "",
         "Appreciation", "Upbeat", "Grunge")

assert_equal(len(test_state.library), 0)
assert_equal(test_state.submission_message,
             "Please fill in all required fields.")
```

These tests check that songs are added correctly and that your validation works.

5. Tests for your page routes

You can also check whether your page routes return a `Page` object.

For example:

Python Run

```
test_state = State(
    library=[],
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="",
```

Python Run

```
    submission_message=""
)

assert_equal(type(index(test_state)), Page)
assert_equal(type(search(test_state)), Page)
assert_equal(type(result(test_state)), Page)
assert_equal(type(recommend(test_state)), Page)
assert_equal(type(about_page(test_state)), Page)
```

This checks that all five page functions return the expected type.

6. Where to put your tests

Put your unit tests at the very bottom of your program, after your `initial_state` definition.

Your current unfinished test:

Python Run

```
test_page = perform_search(test_state, "Isolation", "Angsty", "Pop Punk")
assert_state
```

Can be replaced with the tests above.

A couple of important notes:

- You don't need to test every function equally. Focus especially on `perform_search`, `result`, and `add_song`.
- Your `add_song` function currently checks only the title and artist for blank values. It does not check whether theme, mood, or genre are blank. Your tests should reflect what your code actually does, unless you decide to add that validation.
- You don't need to test `random.choice()` itself. Your tests can use a library with exactly one matching song.

My recommendation: Start with sections 2, 3, and 4. Those tests demonstrate that the actual functionality of your website works, rather than just checking that pages exist. You can use `assert_equal` for all of them!